Comandos do AtomS3R Gateway

Documentação do firmware atual, revisada em 31/10/2026. Inclui as alterações de navegação e intervalo do clima do commit `5c56bcf`. Histórico local: changelog.md; visão de todos os firmwares: catálogo geral.

Como enviar

- **USB serial:** 115200 baud. Envie um comando por linha, terminando com Enter (LF, CR ou CR+LF).
- **UDP:** envie o comando ao IP da placa, porta **4210**. A resposta volta ao endereço e à porta do remetente.
- Respostas longas podem chegar em vários datagramas de até 1200 bytes. O cliente deve continuar recebendo até um período de silêncio; UDP não garante entrega nem ordenação. Em falha local de envio, o firmware reabre a porta e repete o bloco uma vez, sem executar novamente o comando.
- Limite de entrada: **255 bytes** por comando. No serial, linhas maiores ou com NUL são descartadas até a terminação; UDP acima do limite é recusado com erro, sem executar trecho truncado.
- O nome do comando é convertido para minúsculas. Os parâmetros preservam maiúsculas, minúsculas e espaços.
- Use dois-pontos para separar parâmetros. Não acrescente aspas aos exemplos.

Exemplos para o Monitor Serial:

```
help
info
wifi_list
brilho:80
tela:0
clima_sync
```

Sistema e diagnóstico

Comando	Função
`help`	Lista os comandos do firmware. A lista herdada ainda inclui comandos SD, indisponíveis nesta placa.
`name`	Retorna o nome configurado no dispositivo.
`alive`	Responde `alive`.
`info`	Resumo de firmware, rede, memória, data/hora, uptime e reset.
`status`	Estado de Wi-Fi, NTP e heap livre.
`version`	Versão calculada pelo firmware.
`build`	Data e hora da compilação.
`uptime`	Tempo desde a inicialização em dias, horas, minutos e segundos.

Comando	Função
`reason`	Motivo do último reset e uptime atual.
`init`	Consulta o motivo do reset; não reinicializa o dispositivo.
`crash_info`	Consulta o diagnóstico de crash, se core dump estiver habilitado na compilação. Sem SD, não arquiva o relatório e preserva o registro interno.
`cpu`	Modelo, revisão, número de núcleos e frequência da CPU.
`chip_info`	Informações do chip e versão do SDK.
`temp`	Temperatura interna do chip; não é a temperatura ambiente.
`ram`	Estatísticas de heap e uso de RAM.
`heap`	Heap livre em bytes e KB.
`heap_min`	Menor heap livre registrado desde o boot.
`flash`	Informações de flash e espaço do firmware.
`flash_info`	Tamanho da flash, firmware e espaço para OTA.
`health`	Resumo de Wi-Fi, NTP, RAM e atualização do clima.
`selftest`	Verificações de Wi-Fi, NTP, RAM, clima e frequência da CPU.
`reboot`	Reinicia a placa após aproximadamente 500 ms.
`reset_counters`	Zera somente os contadores NVS persistentes de boot/watchdog; preserva as configurações Prometheus.

`health` e `selftest` consideram RAM suficiente a partir de 40 KB de heap livre e clima recente até 30 minutos. Não são testes físicos completos dos periféricos.

Wi-Fi e rede

Comando	Função
`wifi_add:SSID:senha`	Salva ou atualiza uma rede, com confirmação de leitura em NVS. Aplicada na próxima reconexão.
`wifi_list`	Lista os SSIDs salvos, sem as senhas.
`wifi_status`	Estado da conexão, SSID, IP e RSSI.
`net_info`	SSID, IP, gateway, máscara, DNS e RSSI.
`net_monitor`	Estatísticas do monitor de rede e ping ao gateway.
`mac`	Endereço MAC da interface Wi-Fi.
`ip`	IP da conexão Wi-Fi.
`ssid`	Nome da rede conectada.

Comando	Função
`rssi`	Intensidade do sinal em dBm.
`channel`	Canal Wi-Fi atual.
`reset_wifi`	Apaga os perfis Wi-Fi salvos e reinicia.

`reset_wifi` limpa o namespace `wifi`, incluindo o cursor dos perfis, e agenda o reinício para aproximadamente um segundo depois. Preserva os namespaces `ntp_cfg` (fuso/DST) e `telemetry` (Prometheus). Não apaga o firmware.

Há até cinco perfis de rede, com atualização circular. Para rede aberta, mantenha o último dois-pontos:

```
wifi_add:MinhaRede:MinhaSenha
wifi_add:RedeAberta:
```

O primeiro dois-pontos após `wifi_add:` separa SSID e senha: esse formato não permite dois-pontos no SSID. A senha pode conter dois-pontos. Espaços fazem parte das credenciais.

Ping ao gateway mede a resposta do roteador; não comprova acesso à Internet.

Tela e brilho

Comando	Função
`tela:N`	Seleciona uma página, de `0` a `9`.
`rotacao:G`	Gira o display para `0`, `90`, `180` ou `270` graus. Exemplo: `rotacao:90`.
`tela_next`	Avança à próxima tela distinta, pulando páginas duplicadas e o aviso de SD.
`brilho:P`	Define o brilho de `0` a `100` por cento.
`tela_on`	Liga a iluminação com brilho de 80%.
`tela_off`	Define brilho zero; os serviços de rede continuam ativos.

Página	Conteúdo
0	Relógio, IP e temperatura do clima
1 e 2	NTP, data/hora, fuso e DST
3 e 4	Informações de rede
5	Sistema, heap, PSRAM e uptime
6	Cinco redes salvas
7	Aviso de ausência de leitor microSD
8	Monitor de rede
9	Previsão de Porto Alegre

As páginas 1/2 e 3/4 usam o mesmo conteúdo nesta adaptação. O console ocupa a tela por cerca de 10 segundos após um comando; comandos de navegação selecionam a página imediatamente.

Navegação pelo botão (18/10/2026)

Cada toque curto avança uma tela ao soltar: relógio → NTP → rede → sistema → redes salvas → monitor → previsão → relógio. As páginas duplicadas e o aviso de SD são pulados nessa sequência, inclusive por `tela_next`; `tela:N` mantém a numeração da tabela.

Após 20 segundos sem novo toque, a página volta ao relógio. Cada toque reinicia esse prazo; segurar o botão mantém as ações de 3/10 segundos. Selecionar uma página por comando também concede 20 segundos de visualização. O retorno automático não conta como interação para o protetor, que continua entrando após 60 segundos.

Após **60 segundos sem interação**, aparecem dois olhos ciano com movimento suave e piscadas. Botão ou comando sai do protetor. Após dois minutos, o brilho cai para no máximo 20%; uma interação restaura o valor configurado. Se o brilho configurado for zero, use `tela_on` ou `brilho:P` para voltar a iluminar.

Data e hora

Comando	Função
`time`	Consulta a hora local.
`date`	Consulta a data local.
`ntp_status`	Estado de sincronização e data/hora quando disponíveis.
`set_fuso:X`	Define o fuso UTC inteiro entre `-12` e `14`. Exemplo: `set_fuso:-3`.
`dst_on`	Ativa o ajuste de horário de verão.
`dst_off`	Desativa o ajuste de horário de verão.
`set_time:AAAA-MM-DD HH:MM:SS`	Ajusta manualmente a data e hora local.

Exemplo: `set_time:2026-10-17 14:30:00`. O fuso e o ajuste de horário de verão ficam salvos em NVS. O NTP pode atualizar posteriormente a hora ajustada manualmente.

LED RGB

O RGB U5 (NH-B2020RGBA-HF) e controlado pelo LP5562 U4 no endereco I2C 0x30, conforme o esquema oficial AtomS3R v0.4.1. O driver acessa apenas os canais R/G/B, preservando o canal W da iluminacao da tela. O LED verde D1 pertence ao circuito de reset/download e nao e comandado pelo firmware.

Comando	Funcao
`led_on`	Acende o RGB em branco e encerra animacoes.
`led_off`	Apaga o RGB e encerra animacoes.
`led_rgb:R:G:B`	Define a cor; cada componente deve ser inteiro de 0 a 255.
`led_breath`	Pulsacao suave em ciano, sem bloquear o programa.

Comando	Funcao
`led_pisca:P:I`	Pisca P vezes (1 a 100), com intervalo de 1 a 5000 ms entre alternancias.
`led_blink:I`	Pisca continuamente, intervalo de 50 a 60000 ms entre alternancias.

Exemplos: `led_rgb:255:0:0` (vermelho), `led_rgb:0:255:255` (ciano), `led_pisca:5:300`. Sem parametros, `led_pisca` usa 10 pulsos/250 ms e `led_blink` usa 250 ms. `led_on`, `led_off` e `led_rgb` interrompem os efeitos. Comandos manuais tem prioridade sobre a sinalizacao automatica de Wi-Fi. Segurar o botao pode sinalizar a acao de reinicio/apagar redes no RGB.

O boot informa `[RGB] LP5562 pronto (RGB independente do backlight)`. Se houver erro de I2C, os comandos informam indisponibilidade. A resposta do controlador nao substitui a verificacao visual do LED na placa.

`led_blink:I` define o tempo entre alternâncias: um ciclo ligado/desligado leva aproximadamente `2 × I`. Para encerrar um efeito, envie `led_off`, `led_on` ou `led_rgb:R:G:B`; não há comando separado de retorno ao indicador Wi-Fi.

Clima

Comando	Função
`clima`	Temperatura, condição e código WMO do cache atual.
`clima_age`	Tempo desde a última atualização válida.
`previsao`	Data, mínima, máxima, chance de chuva e idade da previsão diária.
`clima_sync`	Solicita uma consulta em segundo plano, se houver Wi-Fi.

Os dados são do Open-Meteo para **Porto Alegre**, herdados do projeto de origem. A primeira consulta ocorre assim que houver Wi-Fi; as consultas automáticas seguintes ocorrem a cada **15 minutos**, inclusive após falha, contados desde a última tentativa. Sem Wi-Fi ou durante OTA, a consulta aguarda a disponibilidade do serviço. `clima_sync` permite antecipar uma consulta e confirma a solicitação, não a conclusão. Consulte `clima_age` e `previsao` para verificar a atualização; `clima` isoladamente não comprova que o cache já foi sincronizado.

SD indisponível

O AtomS3R não tem leitor microSD integrado. Todos os comandos `sd_*` retornam indisponibilidade, inclusive os ainda listados no `help`. `net_history` também retorna SD indisponível. Não há gravação de logs no cartão nem página web de logs.

Os nomes herdados abaixo são **recusados**, não operações disponíveis: `sd_status`, `sd_list`, `sd_read`, `sd_write`, `sd_append`, `sd_del`, `sd_mkdir`, `sd_rmdir`, `sd_log`, `sd_clear_log` e `sd_test`.

Botão e endereços

- Pressão curta no botão frontal: próxima tela distinta ao soltar e saída do protetor; retorno ao relógio após 20 segundos sem novo toque.

- Segurar por 3 segundos e soltar: reinicia.
- Segurar por 10 segundos e soltar: apaga os perfis Wi-Fi e reinicia.
- AP de configuração: **ATOMS3_CONFIG**; página Wi-Fi: `http://192.168.4.1/wifi`.
- Portal conectado: `http://IP_DA_PLACA/`, com `/info`, `/wifi` e `/prometheus` na porta 80.
- Métricas: `http://IP_DA_PLACA:9120/metrics` por padrão; a porta pode ser alterada no portal.
- Hostname OTA: **AtomS3R-Gateway**.

Referência de configuração e bibliotecas: LEIA-ME.md.

Gravacao OTA

Selecione a porta de rede AtomS3R-Gateway na Arduino IDE para atualizar o firmware. A primeira instalacao desta otimizacao ainda usa o firmware anterior; as otimizacoes passam a valer no boot seguinte.

A OTA recebe atendimento antes do portal e das animacoes. Durante a transferencia, a biblioteca atende exclusivamente a gravacao na tarefa principal: a tela mostra barra e porcentagem de progresso; botao e portal ficam temporariamente sem atualizacao. Novas consultas de clima sao impedidas; uma consulta em andamento pode terminar. O Wi-Fi fica sem economia de energia, a CPU usa 240 MHz e a prioridade da tarefa aumenta temporariamente. Frequencia e prioridade retornam aos valores anteriores em caso de erro; o Wi-Fi continua sem economia de energia para permitir nova tentativa rapida.

Na tela, a barra avança em passos de pelo menos 5 pontos percentuais, com intervalo mínimo de 250 ms; a chegada a 100% é exibida imediatamente. Durante a validação aparece “Verificando...”; o sucesso mostra “Concluido!” e falhas mostram o código de erro. O serial informa progresso a cada aproximadamente 10% e termina com bytes, segundos e KiB/s. Compare esse registro entre gravacoes para medir a taxa real. O tempo depende do sinal Wi-Fi, roteador, computador e tamanho do firmware; nao foi medida uma aceleracao na placa nesta revisao.

Olhos e acelerometro

A carinha possui boca e sobrancelhas ciano, com expressões feliz, curiosa, surpresa e sonolenta. Elas mudam automaticamente a cada 3,5 a 6,5 segundos; um movimento forte da placa provoca surpresa por cerca de dois segundos. Não é necessário comando adicional.

No protetor, os dois olhos ciano acompanham a inclinacao e aceleracao do AtomS3R. A posicao ao entrar no protetor e usada como referencia neutra. As leituras possuem suavizacao e uma zona morta para evitar tremores; as piscadas continuam automaticas. Mexer a placa nao sai do protetor: use botao ou comando. Se a IMU falhar, os olhos voltam ao movimento automatico.

O serial informa `[IMU] Acelerometro pronto: olhos seguem o movimento` quando a inicializacao foi bem-sucedida. O sensor nao cria um comando novo; basta aguardar os 60 segundos de inatividade e inclinar a placa.

Limites e cuidados

- UDP e configuração web não oferecem autenticação; use somente rede confiável.
- O firmware atual não configura senha com `ArduinoOTA.setPassword()`. Não presuma que a senha usada em outro projeto ESP32 também se aplica aqui.

- `crash_info` depende do suporte de core dump da compilação. Sem SD, a falha de arquivamento preserva o dump; guarde o ELF correspondente ao crash.
- `init` não reinicia, `clima_sync` não espera a consulta terminar e `tela_off` não coloca a CPU em deep sleep.
- Esta revisão documental não confirma compilação completa, gravação ou teste físico das alterações locais.